

Open in app ↗

Sign up

Sign in

Medium

 Search數學、人工智慧與... · [Follow publication](#)

SMOTE + ENN : 解決數據不平衡建模的採樣方法

17 min read · Oct 27, 2019



Edward Tung

[Follow](#)

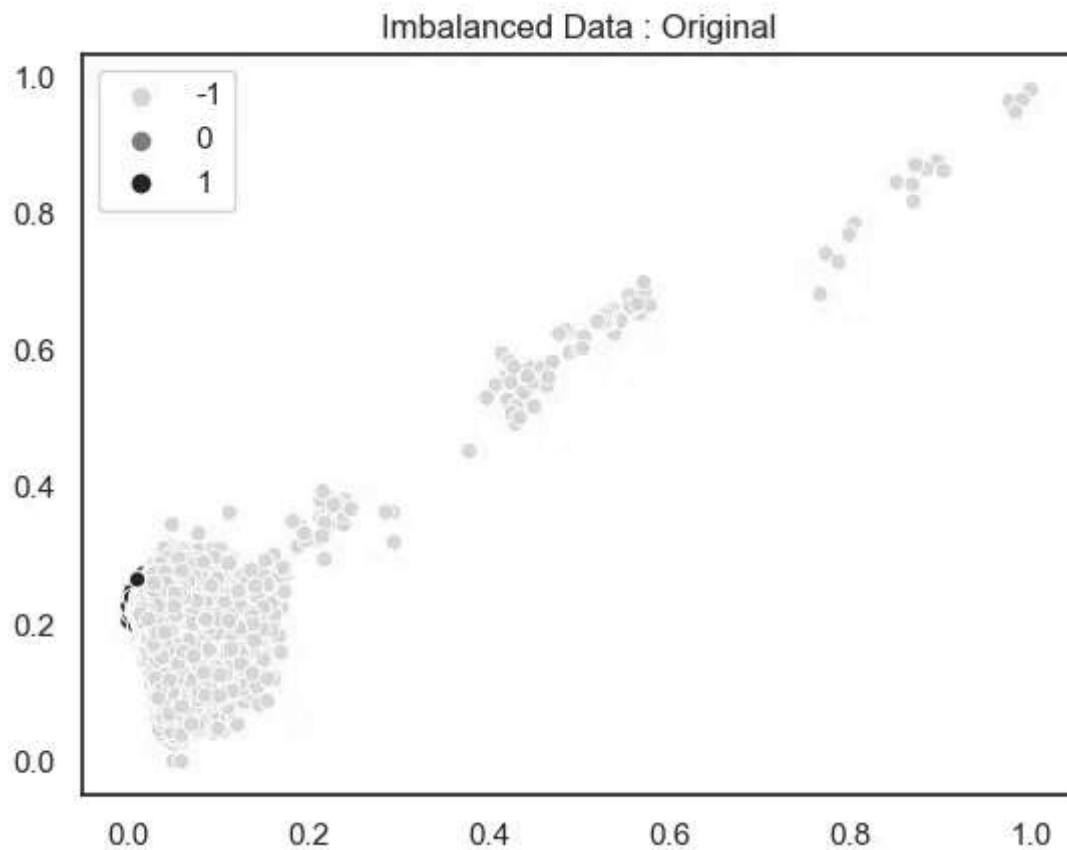
Share

SMOTE + ENN : A sampling method that solves modeling with an imbalanced dataset

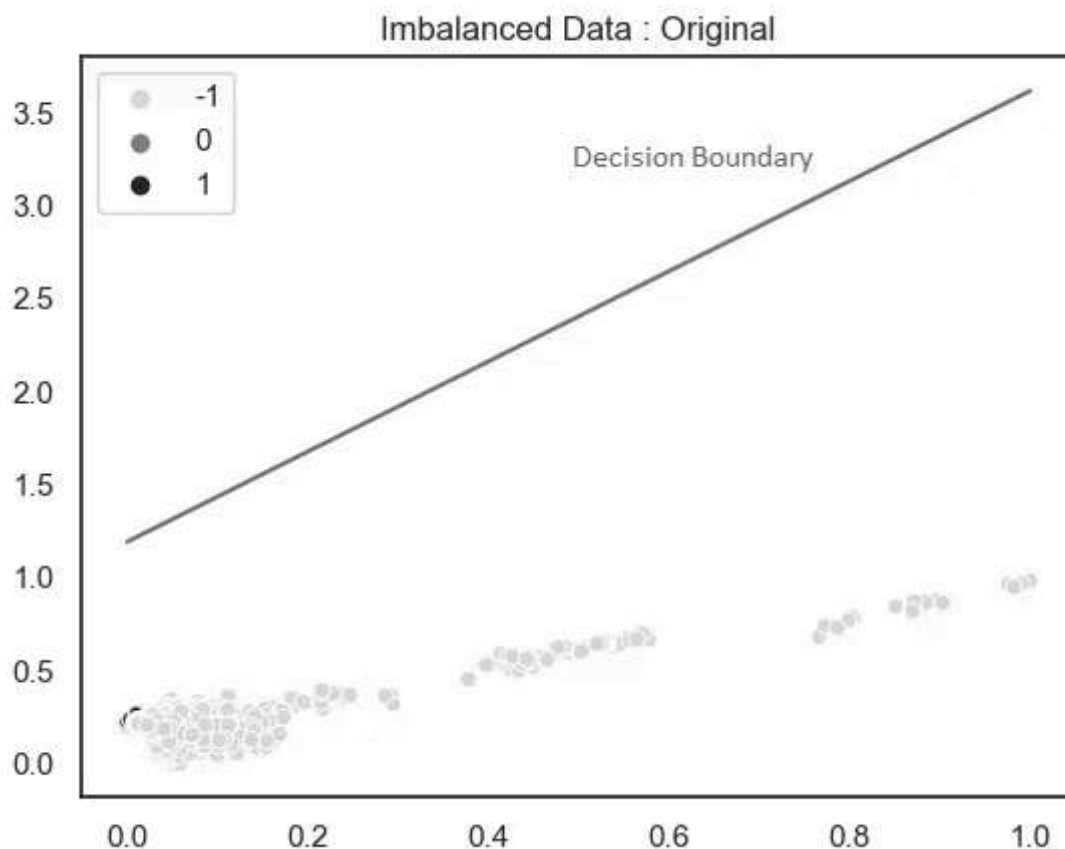
前言

工作使人衰老...好久沒寫文章了，這次趁案子輪替的空閒補上一篇！

我們知道，在數據科學的領域中一個很重要的環節就是分類建模，舉凡簡單的邏輯回歸，到更複雜的隨機森林、深度學習方法等。然而，真實世界的資料常常遇到不可控的問題，使得建模效果急速下降，一個很常見的例子就是數據不平衡的問題，少數樣本與多數樣本常有很大的落差，舉例而言，廣發簡訊詢問消費者願不願意購買某項產品，可能測試了數萬人至數十萬人的規模，卻僅有不到一千人回覆願意，此時的建模效果就會變得極差，當然我們可以透過更加集成學習、深度網絡的方法去強硬地提升準確度，但相對地必須承擔過擬合的風險，如下圖，假設我有一筆資料集長得像這樣：



上面展示了一個不平衡數據集，其中只有 0.8% 是正樣本。假設我們硬是要建模，決策邊界就會長得很可怕 (這邊我都用線性的模型來做展示，因為若是單純線性能夠有效果，那麼集成的模型也會是同樣道理)：



如上圖所示，可以看到決策邊界根本沒能力區分出正樣本，但實際上模型有非常不錯的準確率，這是因為模型只要將所有樣本全部預測為負樣本(這裡將多數樣本視為負樣本，少數樣本視為正樣本)，就可以得到很好的效果。

順手附上測試用代碼 (*不知為何用Gist畫面會被裁掉，先用截圖替代，之後復活了再補上)：

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.model_selection import train_test_split
6 from sklearn.decomposition import PCA
7 from sklearn.linear_model import LogisticRegression
8 from sklearn.metrics import roc_auc_score, classification_report
9 from sklearn.preprocessing import MinMaxScaler
10 from imblearn.datasets import fetch_datasets
11 sns.set(style='white')
12
13 df = fetch_datasets()['protein_homo']
14 X = PCA(n_components=2).fit_transform(df.data)
15 X = MinMaxScaler().fit_transform(X)
16 Y = df.target
17
18 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.33, random_state=42, shuffle=True)
19
20 print('X_Shape : ', X.shape)
21 print('Y_Shape : ', Y.shape)
22 print('Positive Ratio : ', np.count_nonzero(Y == 1)/Y.shape[0])
23
24 lr = LogisticRegression().fit(X_train, Y_train)
25 Y_pred = lr.predict(X_test)
26
27 print('Report : ', classification_report(Y_test, Y_pred))
28 print('ROC : ', roc_auc_score(Y_test, Y_pred))
29
30 plot_base = np.linspace(0, 1, 1000)
31 value = (-lr.coef_[0][0] * plot_base - lr.intercept_) / lr.coef_[0][1]
32
33 sns.scatterplot(X_train[:, 0], X_train[:, 1], hue=Y_train)
34 plt.plot(plot_base, value)
35 plt.title('Imbalanced Data : Original')
36 plt.ylim(0, 1)
37 plt.show()
```

這樣的問題該怎麼解決呢? 我們希望模型能更準確，同時又不希望只是不斷深度學習或加深決策樹的層級，來造成可怕的過擬合，於是乎，統計學家們提出了幾種思路：

一、透過更好的方式衡量模型：思路很簡單，我們至少得先知道這個模型效果不好，後續才有辦法找修正方向，如果一開始就認為模型很厲害，就將模型應用下去，會造成難以挽回的後果。這邊介紹三種比較常用的指標，混淆矩陣、準確/精確/召回率、F1分數以及ROC分數，當然我們要再透過測試/驗證集與交叉驗證等方法去準確評估。

二、對資料做一些處理：這邊思路有兩種，一種是將多數樣本中比較不具代表性的拿除掉，以免造成雜訊，稱為欠採樣，第二種是將少數樣本用某種方式重複抽樣或合成新樣本，稱為過採樣；本文除了介紹這兩種方法以外，也會嘗試將這兩種方法結合，並配合一般建模流程提出完善的解決方案。

另外，本文中的一些算法由於過於複雜，這邊會使用 python imblearn 這個套件來協助做一些簡單的抽樣，參考文檔如下：

Welcome to imbalanced-learn documentation! - imbalanced-learn 0.5.0 documentation

Information to install, test, and contribute to the package. The main documentation. This contains an in-depth...

imbalanced-learn.readthedocs.io

二分類模型的評估指標

首先我們從評估指標開始，一般來說準確度是一個直覺性最高的指標，但如上一小節提到的問題，單純看準確度沒辦法精準衡量模型是好是壞，因此這裡介紹幾種更常見的評估方式：

【混淆矩陣 Confusion Matrix】

我們如果將上一小節二分類問題的結果，用這樣的方式來看：

	True Positive	True Negative
Predicted Positive	0	0
Predicted Negative	398	47700

這樣的方法叫做混淆矩陣，這也是為甚麼我們說不要透過準確率來判斷模型好壞的原因，上面的結果準確率達到驚人的 $47700 / (47700 + 398) = 99.2\%$ ，乍看之下似乎不錯，然而如上圖所示，正樣本的正確分類率是 0%，意味著模型根本沒辦法辨別出正樣本的存在。

這樣的表示方式分成四大塊：

1. True Positive 真陽性：實際上是正樣本且被預測為正樣本
2. True Negative 真陰性：實際上是負樣本且被預測為負樣本
3. False Positive 假陽性：實際上是負樣本但被錯誤預測為正樣本
4. False Negative 假陰性：實際上是正樣本但被錯誤預測為負樣本

我們平時說的準確度 Accuracy Rate，就是(真陽性 + 真陰性) / (所有樣本)的公式得出的，可以衡量整體而言的正確分類率是多少。

【精確度與召回率 Precision and Recall】

有了混淆矩陣的概念之後，我們進一步得出兩個指標，精確度與召回率。

精確度 Precision 的公式為 $TP / (TP + FP)$ ，意思是所有被預測為正確的樣本中，有多少比例實際上是正確的。召回率的公式則是 $TP / (TP + FN)$ ，也就是說在實際上是正確的樣本中，有多少比例是被預測為正確的。

這兩個指標更加關注了實際上的正樣本，精確率隱含的意思是我們希望模型的預測結果更精確，召回率的意思則是希望正樣本能盡可能被偵測到。

【F1 分數 F1-Score】

那麼，怎麼綜合來看精確度與召回率呢？

我們定義一個分數叫做 F1 分數，計算公式為 $(2/F1-Score) = (1/Precision) + (1/Recall)$ ，經過代換之後則是 $F1-Score = 2 * Precision * Recall / (Precision + Recall) = 2 * TP / (2 * TP + FP + FN)$ 。聽起來很複雜，觀念上卻很簡單，在大多數情況下，我們並不會只關注精確度或召回率，而是希望他們分配合理的權重，以F1-Score的例子而言，我們認為權重應該一樣。

F1 分數公式設計的原因在於，經過這樣的轉換以後，F1 分數的值會在 0 與 1 之間，便於評估。此外，上面提到的權重問題實際上也可以被調整，人們有時候認為精確率和召回率的權重應該不同，因此提出了：

$$F_{\beta} = \frac{(1 + \beta^2) * PR}{P * \beta^2 + R}$$

假設 $\beta = 0.5$ ，則我們認為召回率的重要性是精確率的一半，如果 $\beta = 2$ ，則召回率的重要性是精確率的兩倍，大家可以自己帶入數字看看。

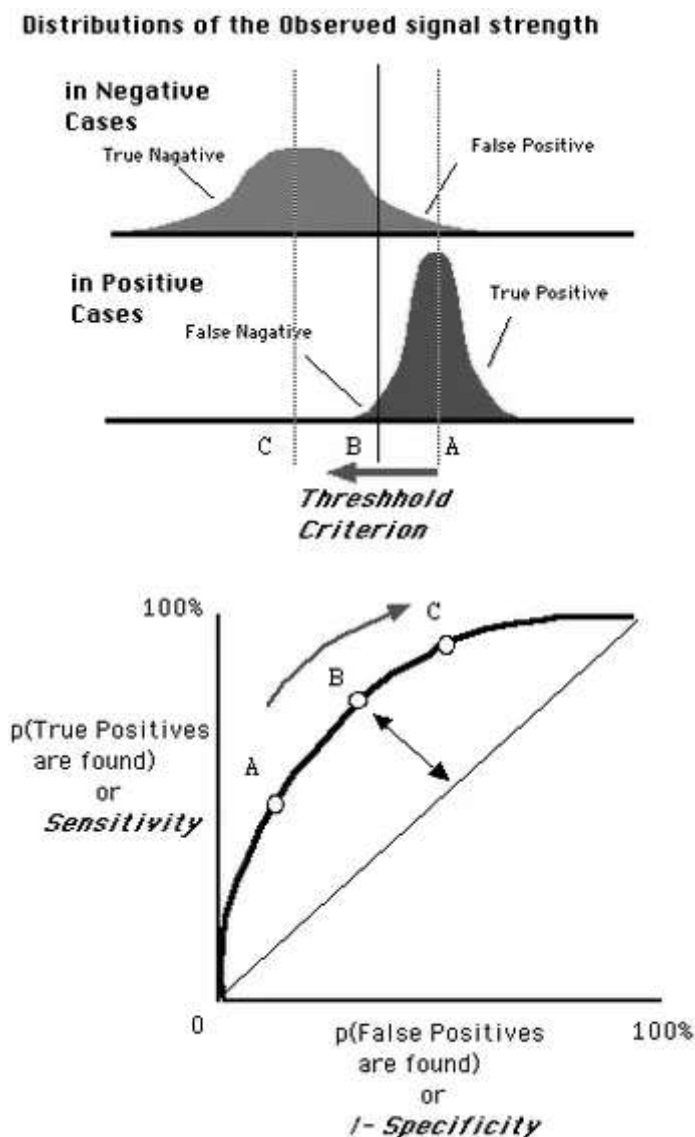
【ROC 分數 / 曲線】

我們進一步提出ROC這個概念來衡量模型好壞，首先我們先定義兩個指標：

1. $TPR = TP / (TP + FN)$ ，所有正樣本中被正確判斷為正樣本的比例 (實際上就是 Recall 的意思)
2. $FPR = FP / (FP + TN)$ ，所有負樣本中被錯誤判斷為正樣本的比例

有了這兩個指標以後，配合模型的閾值(threshold)，我們就可以畫出一條ROC曲線，閾值是甚麼意思呢？一般我們在使用二分類模型的時候，會將預測出來的機率值設定一個門檻，高過這個門檻的預測為正，低於門檻的預測為負，如果閾值設定為0.5，意思是說模型預測結果為 0.5 以上的時候，輸出的結果就是正樣本，反之則為負樣本。

閾值的範圍從 0 到 1，而曲線的畫法就要考量剛才說的兩種指標，如果我們將橫軸設定為 FPR，縱軸設定為 TPR，配合不同的閾值就可以畫出一條曲線，我們這邊用 Wikipedia 的示意圖來了解一下：

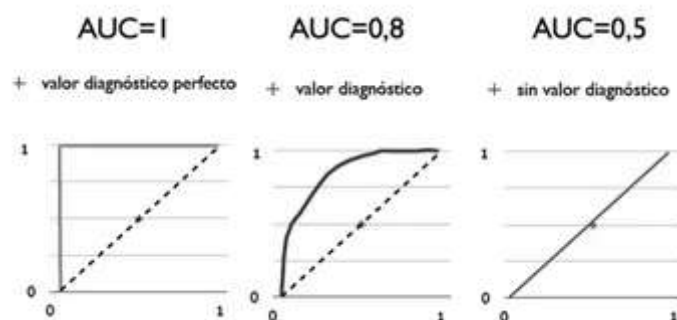


Source : Wikipedia

首先看圖的上半部，藍色分布是負樣本，紅色分布是正樣本，橫軸則是模型預測的機率，簡單來說，在模型預測出來的機率上，正樣本與負樣本的分布是有差異的，正樣本集中在模型預測機率高的分段上，負樣本則較低，此時，我們可以設定一個閾值，也就是上面的 Threshold Criterion，比方說選擇 B 點作為閾值的時候，你可以看到大部分正樣本都被預測進去，但有少部分負樣本也被預測為正樣本。

結合不同的閾值，我們就可以畫出圖的下半部曲線，從左到右分別是把機率由高到低排序，當我們設定機率很高的時候，幾乎沒有辦法預測成功，當我們將機率稍微下調到 A 點，可以看到 TPR 跟 FPR 有顯著的變化，大約有一半的正樣本被成功預測為正樣本，TPR 大約等於 0.5，而有少部分的負樣本被預測為正樣本，FPR 大約等於 0.2，因此得到了 A 點在曲線上的位置。

甚麼叫做一個好的模型呢？我們就提出了 ROC 分數這個概念，或者更常叫它 AUC (Area Under The Curve)，也就是曲線下的面積，曲值從 0.5 到 1，如果這個分數為 0.5，代表曲線基本等於對角線，模型沒有任何鑑別度，因為不管你正樣本分對比例多高，永遠都會有同等比例的負樣本被錯判。而如果分數為 1，恭喜你這是一個完美分類的模型，因為當你 100% 成功預測出正樣本的時候，沒有任何的負樣本被錯誤地分類。



Source : Wikipedia

參考上圖，左邊的圖是最好的，呈現一個三角形，右邊的則是最差，如果我們把上面小樣本的模型放進去，你也會看到類似的曲線，事實上在上面的範例代碼中，我已經透過 `sklearn.metrics` 中的套件 `roc_auc_score`, `classification_report` 兩者去將所有的指標都給 `print` 出來了。可以動手嘗試一下，你會發現 ROC 分數很可悲地只有 0.5。

講到這裡，我們至少可以用更科學的方法來判斷模型好壞，但實際問題仍沒有解決，樣本比例差異如此大的情況下，總是造成模型判斷能力差的問題，因此，下面開始我們要透過一系列的採樣方法來直面這個難關。

過採樣方法 : Synthesized Minority Oversampling Technique (SMOTE)

最先出現在我們思路的想法是，怎麼樣把少數樣本變多？這個做法我們稱之為過採樣，常見的做法有幾種，把少數樣本重複抽樣出來、用某種分布的方式去抽取樣本，或是用人工手段合成一些樣本。

最簡單的作法當然就是直接複製一些少數樣本，或是根據少數樣本的某種分布去採樣某些樣本，根據樣本的採樣可以看我之前寫的系列一：

抽樣與蒙地卡羅(一)：隨機數生成

Sampling and Monte Carlo (01) : Generating Random Number

medium.com

* 還欠著沒寫完，之後補上蒙地卡羅的部分

這樣的好處當然就是簡單、好懂，但壞處也很明顯，就是容易過擬合，你可以想像盡管模型分數表現再好，終究只是根據同一組樣本的訊息去學習，如果今天出現了一些跟少數樣本很接近，但又不完全符合少數樣本特徵的資料點，模型很可能就會失效了。

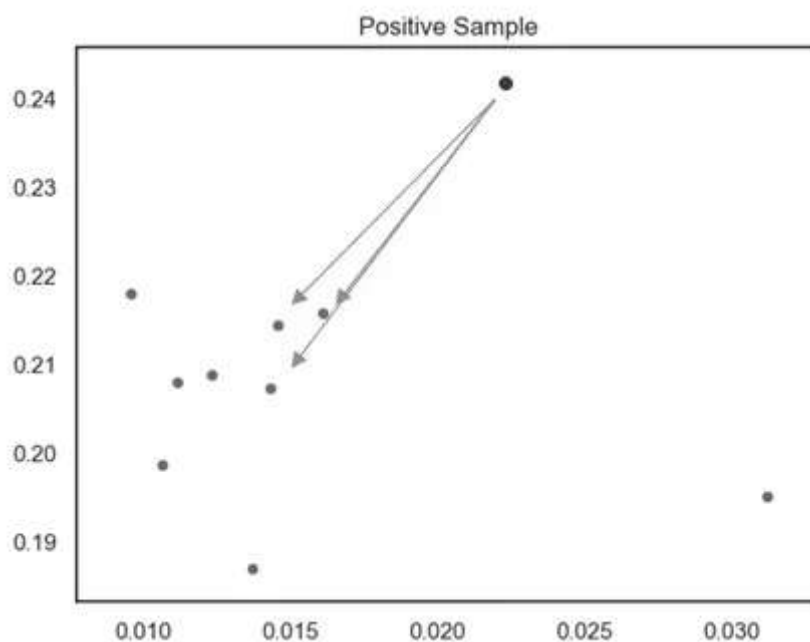
【SMOTE 方法：合成少數過採樣方法】

我們引進了新的方法叫做 SMOTE 方法，這是 2002 年提出的一篇論文，主要概念也就是在少數樣本位置近的地方，人工合成一些樣本，整個算法的流程如下：

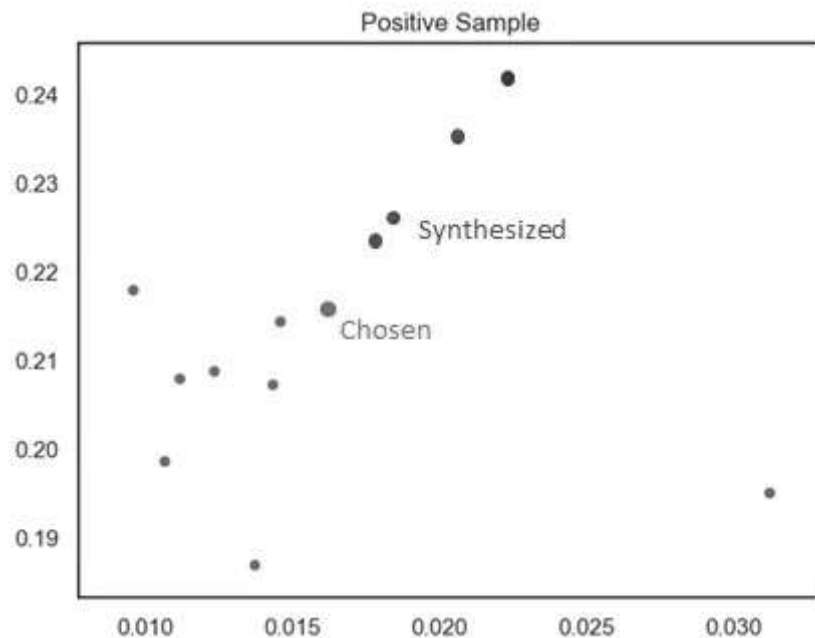
1. 設定一個採樣倍率 N ，也就是對每個樣本需要生成幾個合成樣本
2. 設定一個近鄰值 K ，針對該樣本找出 K 個最近鄰樣本並從中隨機選一個
3. 根據以下公式來創造 N 個樣本

$$x_{new} = x_{chosen} + (x_{nearest} - x_{chosen}) * \delta; \delta \in [0, 1]$$

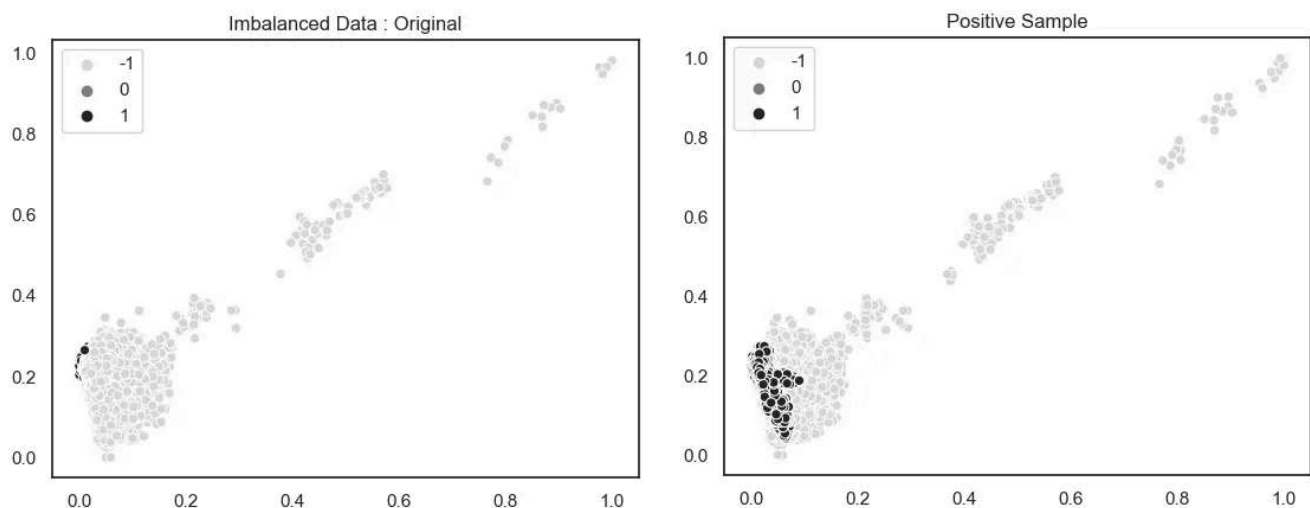
我們用圖的方式一步一步來看，首先，我們隨機選擇一個少數樣本點，並找出他的 K 個近鄰點，這裡我們將 K 設定為 3：



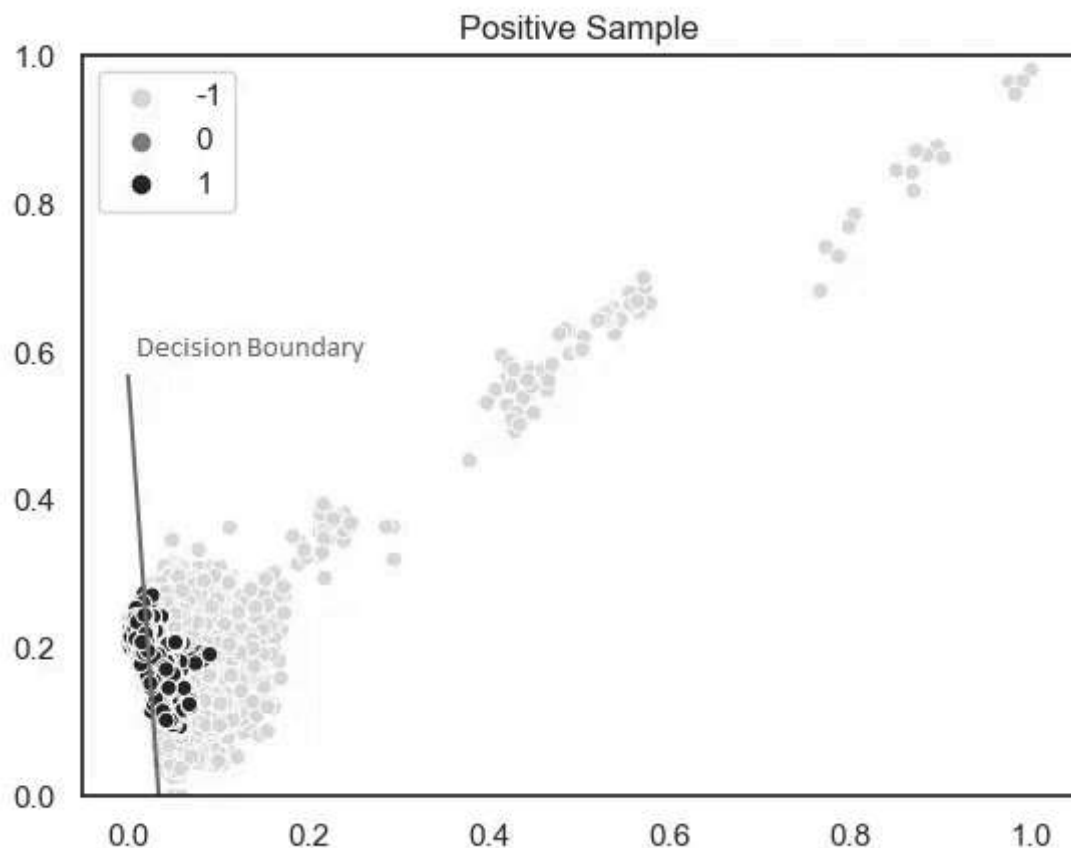
從三個近鄰點中隨機選取一個，並透過公式去合成 N 個樣本點，這裡我們也將 N 設定為 3：



如上圖所示，我們合成了新樣本，接著只要對所有點做一樣的事情即可，我們透過一開始的圖來做個比較看看：



右邊是透過 SMOTE 方法採樣得到的新樣本，可以看到很明顯比左邊的要好一些，我們接著用這個資料集建模看看：



可以看到決策邊界比起之前好了很多，這個模型的 ROC 分數也來到了 0.72，當然這是在測試集上的分數，因此我們可以說該模型初步具備了一定的鑑別能力，這邊附上測試的代碼，使用的是一開始提到的 imblearn 套件 (自己照著算法刻也是可以，但因為算法不複雜但代碼很複雜，這邊就不一步一步來了，關鍵是理解演算法後面的意涵)：

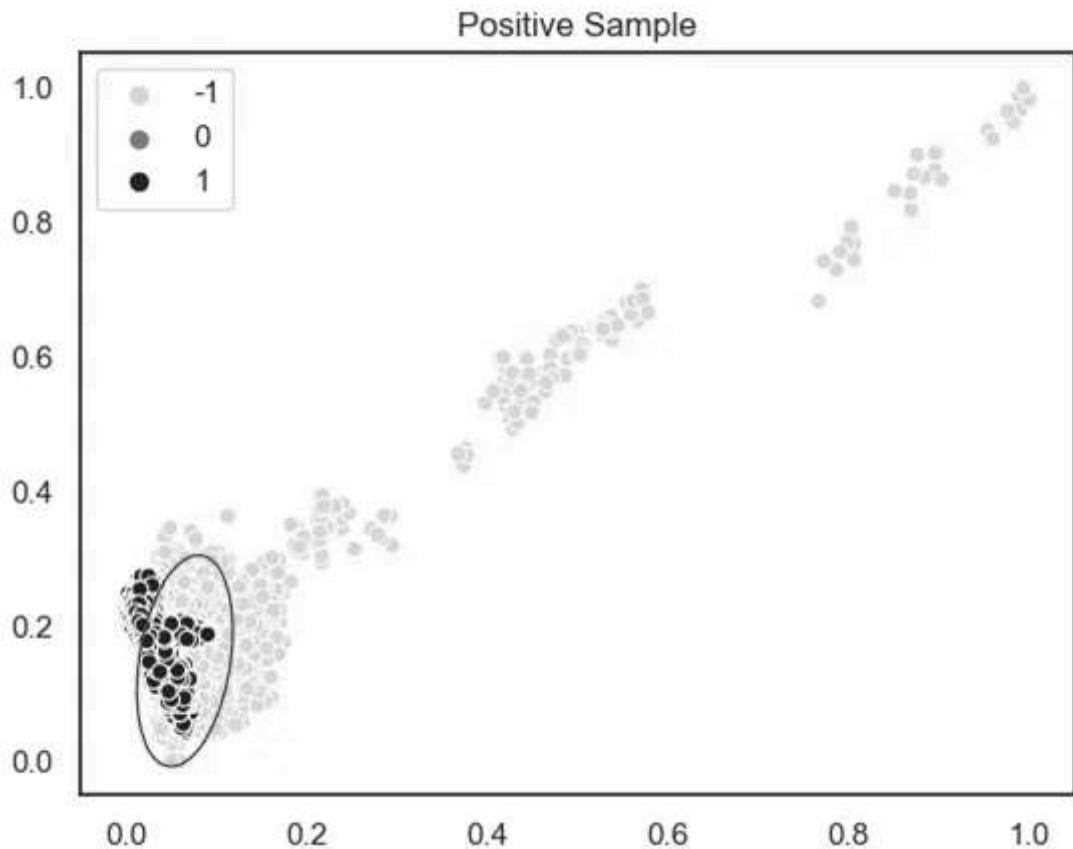
```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from imblearn.datasets import fetch_datasets
6 from sklearn.preprocessing import MinMaxScaler
7 from sklearn.model_selection import train_test_split
8 from sklearn.decomposition import PCA
9 from imblearn.over_sampling import SMOTE
10 from sklearn.linear_model import LogisticRegression
11 from sklearn.metrics import roc_auc_score, classification_report
12 sns.set(style='white')
13
14 ### Data Preparation ###
15 df = fetch_datasets()['protein_homo']
16 X = PCA(n_components=2).fit_transform(df.data)
17 X = MinMaxScaler().fit_transform(X)
18 Y = df.target
19 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.33, random_state=42, shuffle=True)
20
21 ### SMOTE 方法 ###
22 X_res, Y_res = SMOTE(random_state=42).fit_resample(X_train, Y_train)
23
24 ### 建模 ###
25 lr = LogisticRegression().fit(X_res, Y_res)
26 Y_pred = lr.predict(X_test)
27 print('Report : ', classification_report(Y_test, Y_pred))
28 print('ROC : ', roc_auc_score(Y_test, Y_pred))
29 plot_base = np.linspace(0, 1, 1000)
30 value = (-lr.coef_[0][0] * plot_base - lr.intercept_) / lr.coef_[0][1]
31
32 ### 畫圖 ###
33 sns.scatterplot(X_res[:, 0], X_res[:, 1], hue=Y_res)
34 plt.plot(plot_base, value)
35 plt.title('Positive Sample')
36 plt.ylim(0, 1)
37 plt.show()
```

同樣：請原諒我先用這個方式呈現

** 當然，如果要更精準還可以做交叉驗證，但這邊僅做展示就不多作了

【Border Line SMOTE 方法】

上面所提到的方法雖然不錯，但有一個很明顯地缺點，那就是對於所有少數樣本都會做過採樣，然而，大多時候並不是所有少數樣本都失去鑑別度，真正沒有鑑別度的是那些跟多數樣本混合在一起的少數樣本，如下圖：

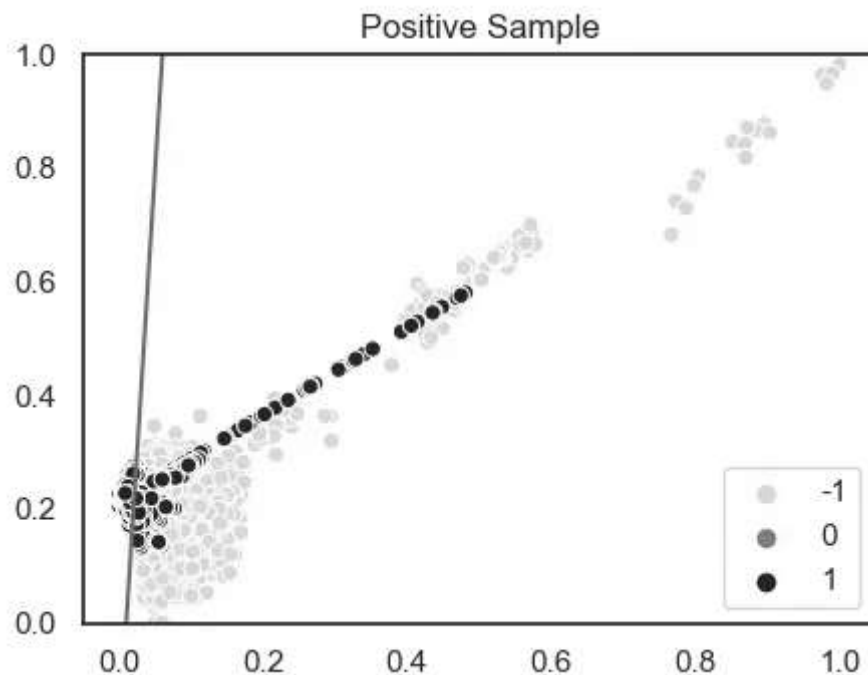


相對而言，靠近邊界的少數樣本因為容易跟多數樣本混合在一起，因此容易產生噪音，也就是我們對這些樣本並沒有這麼有信心，這很容易理解，因為如果我們對這些邊界的樣本進行學習，很有可能我們會將多數樣本誤判為少數樣本，此時我們就可以對算法做出改進，變成 SMOTE Border Line 算法：

1. 在所有少數樣本點中，對整個樣本搜索最近鄰 (相比於原始的 SMOTE 僅對於其他少數樣本搜尋)
2. 這些最近鄰中，少數樣本的數量為 x ，多數樣本的數量為 y ， $K = x+y$ 即我們輸入的參數，也就是要尋找多少個最近鄰
3. 設定參數 α ，這個 α 會被拿去與 $x / (x+y)$ 做比較，如果 α 比較大，說明跟我們設定的門檻 α 比較起來，少數樣本的比例較低，換言之更有可能該少數樣本是邊界樣本，我們把他畫入 Danger Set
4. 當然，也可以設定另一個參數，當少數樣本比例過低，我們就直接認為該樣本為 noise，不對他進行任何操作；此外，少數樣本比例過高的地區，我們也直接認為其為安全樣本，也不用進行合成。
5. 最終，我們只對 Danger Set 的樣本去取樣

代碼上，只需修改一行即可：

```
X_res, Y_res = BorderlineSMOTE(random_state=42, kind='borderline-2').fit_resample(X_train, Y_train)
```



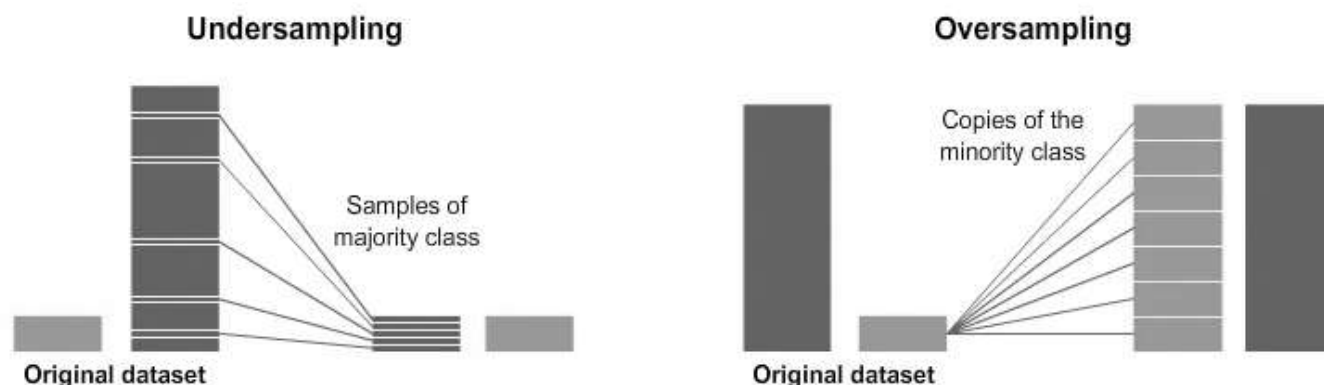
重新採樣的結果如上圖，ROC 來到了 0.73，模型的確是有進步的，那一排跑掉的斜線估計是少數樣本點中有神奇的離群值存在。

當然，實務上還是更常見一般的SMOTE方法，畢竟Borderline方法的計算複雜，閾值的設定也缺乏公定的標準，相對來說需要一些力氣調參，而模型進步的幅度不大，是主要不採納的原因。

整體而言，SMOTE方法雖然看似科學一些，但畢竟是對少數樣本去合成，還是難免有過擬合風險，只是相對於完全沒有鑑別度的模型而言，我們通常寧願放棄一些風險去進行過採樣，你也可以透過交叉驗證去減少這個風險。此外，SMOTE 方法對樣本的合成是線性地，這是來源與手寫數字識別，對數字做旋轉、拉伸，實際上並不會改變數字的長相，因此做線性插值是對樣本合成一個相對理想的作法，然而在資料的世界中，這件事並不完全有道理，譬如在資料並非以歐幾里得空間分布來呈現時，線性插值往往是另一種噪聲，因此使用時必須對資料進行許多觀察才行。

欠採樣方法 : Edited Nearest Neighbor

相對於過採樣，欠採樣是將多數樣本進行 Scale Down，使得模型的加權權重改變，少考慮一些多數樣本，這張圖很好地展示了兩種方法的差異：



Source : Kaggle

同樣，最簡單的做法就是隨機排除掉一些多數樣本，但不推薦的原因是有可能誤排除掉邊界樣本，使得模型鑑別度雖然上升，卻大量增加過擬合風險，因為邊界沒有鑑別度的少數樣本也被模型考慮進來，為了盡可能解決這個問題，這邊介紹兩種方法：

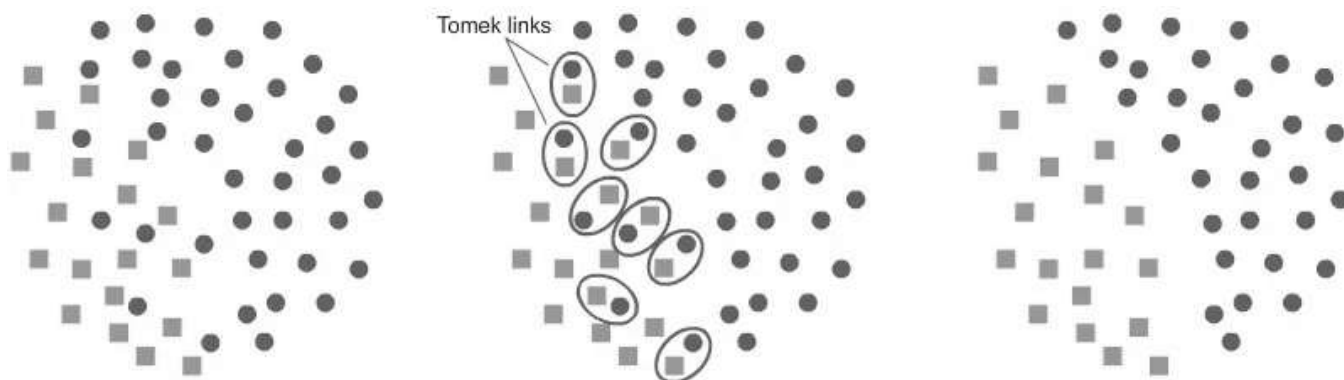
【Tomek Link】

*For an instance pair $(x_i, x_j); x_i \in S_{max}; x_j \in S_{min}$
 If there is no x_k such that :
 $d(x_i, x_k) < d(x_i, x_j)$ or $d(x_j, x_k) < d(x_i, x_j)$
 Then (x_i, x_j) is a tomek link*

Tomek Link 定義

Tomek Link 算法會針對所有樣本去遍歷一次，假設今天有兩個樣本點 x, y 分屬不同的 class，也就是一個為多數樣本一個為少數樣本，我們可以去計算它們之間的距離 $d(x, y)$ 。

此時如果我們找不到第三個樣本點 z ，使得任一樣本點到 z 的距離比樣本點之間的距離還小的話，我們就稱為 Tomek Link，我們先來看一下圖幫助理解：



Source : Kaggle

Tomek Link 的關鍵思路在於，找出邊界那些鑑別度不高的樣本，與 Borderline SMOTE 有點像，認為這些樣本點屬於雜訊，應該剔除，因此可以見上圖最右邊，剔除以後兩種類別的樣本點可以更好地區分開來。

同樣，代碼也僅需修改兩行即可：

```
from imblearn.under_sampling import TomekLinks
X_res, Y_res = TomekLinks().fit_resample(X_train, Y_train)
```

【Edited Nearest Neighbor，ENN 算法】

與上面 Tomek Links 的觀念相同，也是透過某種方式來剔除鑑別度低的樣本，只是這邊的方式改成了對多數類的樣本尋找 K 個近鄰點，如果有一半以上(當然，門檻可以自己設定)都不屬於多數樣本，就將該樣本剔除，通常這些樣本也會出現在少數樣本之中。

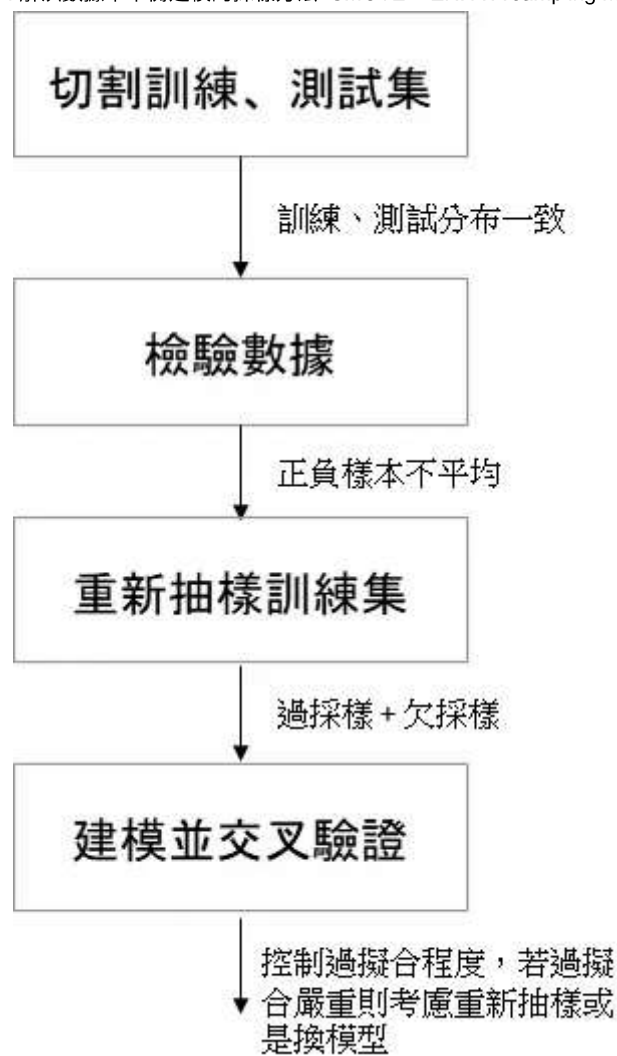
【結合過採樣與欠採樣算法】

當然，上面講了這麼多的算法並不是不能重疊再一起的，我們大可以使用兩者的結合，比方說 SMOTE + ENN、SMOTE + Tomek Links 算法等等，具體哪個效果最好則考量資料的分布情況以及多方嘗試幾組參數。

當然，採樣方法還有更多的配合來改善模型效果，比方說透過 Ensemble 的方式讓欠採樣的模型去做集成，一種 EasyEnsemble 的思路就是透過組合許多 Scaled Down 的數據集來做加權組合，不過 Ensemble 方法往往會讓模型流失掉許多直觀的解釋性，我們這邊就不多談了。

結尾：不平衡數據集分類建模流程

最後，我總結了一下不平衡數據分類的建模流程：



有幾個值得注意的點：

1. 應該先切分測試、訓練集，再針對訓練集去過採樣或欠採樣，這是因為重新採樣的目標是要讓模型產生鑑別度，而不是讓模型學習到錯誤的樣本，如果你先採樣再去切分，很有可能你的測試集以及偏離了原本的資料樣貌，導致該模型學習到一堆雜訊
2. 要透過交叉驗證去控制過擬合，原因在於不管用哪種採樣方式，幾乎都會增大過擬合程度，尤其是在樣本數少，又做欠採樣的情況下，可能模型雖然區分出來了，但是因為欠採樣後多數樣本過少等等問題，造成模型只側重學習某部分的樣本，無法反映全局情況，此時交叉驗證甚至建立多個模型去做集成都會是好的方式
3. 觀察少數樣本與多數樣本分布情形，本文舉的例子是因為少數樣本與多數樣本看上去還能分離，實際運行很有可能碰到完全分不開的例子，少數樣本雜亂地散落在多數樣本之間，此時就不要考慮太多採樣問題，反倒可以優先評估是否資料本身的分佈有問題，像是一開始回收數據錯誤，或是樣本並非歐幾里得分布等等的情形

到這邊就算是介紹完了當今比較常見的對數據不平衡時的採樣方式，希望對各位讀者有所幫助。此外工作以後確實較少時間可以繼續研讀，也就沒有時間好好整理完之後再寫成文章，畢竟一篇文章從蒐集材料、構想到花點時間實作也是十幾個小時不見了，因此還請大家多多包涵。

Data Science

Python

Machine Learning

Sampling

Statistics



Follow

Published in 數學、人工智慧與蟒蛇

278 followers · Last published Mar 22, 2021

Recording some interesting data science topics here :)



Follow

Written by Edward Tung

1.6K followers · 59 following

Columbia Student || 2 yrs of data scientist and 1 yr of business consultant experience

Responses (3)



Write a response

What are your thoughts?



Sandychun

Jul 26, 2024



您好：

抱歉冒昧打擾，我是《面試經驗談》與《科技島》網站小編。《面試經驗談》是個專為找工作、解決求職者面試問題的網站；而《科技島》目的，是為提供最新科技資訊給求職者、進而找到好工作。

兩個網站均設有針對職務心得、工作技巧、從業所得提供經驗分享單元，讓現正從事科技業或未來想進入科技業的學弟妹們可以更加瞭解這個行業。

剛剛瀏覽到您的大作，發現對正在找工作的網友們很有幫助，不知這篇【SMOTE + ENN ... [more](#)

[Reply](#)

Minchi Chen

Aug 31, 2023



我是初學者，謝謝你深入淺出的介紹。

請問最後一段 "值得注意的點"2 中提到 "要透過交叉驗證去控制過擬合"的意思，是否指 (例如用random forest classifier):

1. 對SMOTE後的訓練集做 cross validation, 得到 F1, ROC, precision....
2. 用以上 random forest model 去預測 test dataset 所得的 F1, ROC, precision..
3. 若 1 的 F1, ROC, precision... [more](#)

[Reply](#)

yshuang

Oct 27, 2019



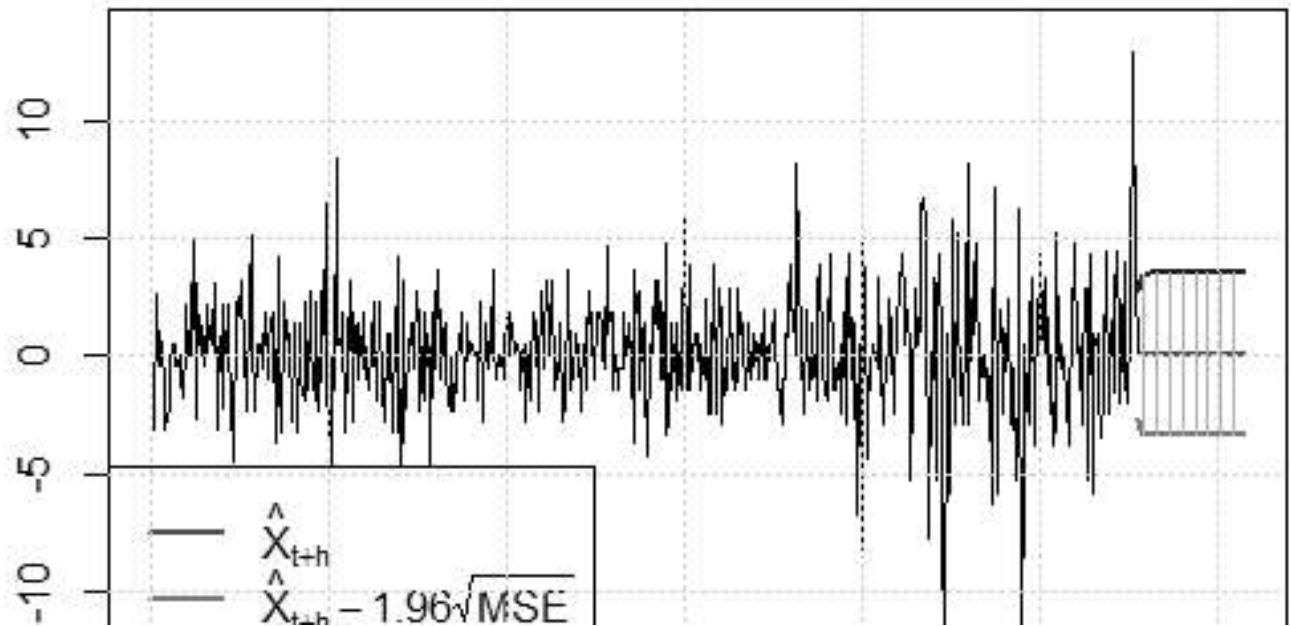
乾原來是因為我先SMOTE才切測試集，難怪跑起來怪怪的，受教受教



1 reply

[Reply](#)

More from Edward Tung and 數學、人工智慧與蟒蛇

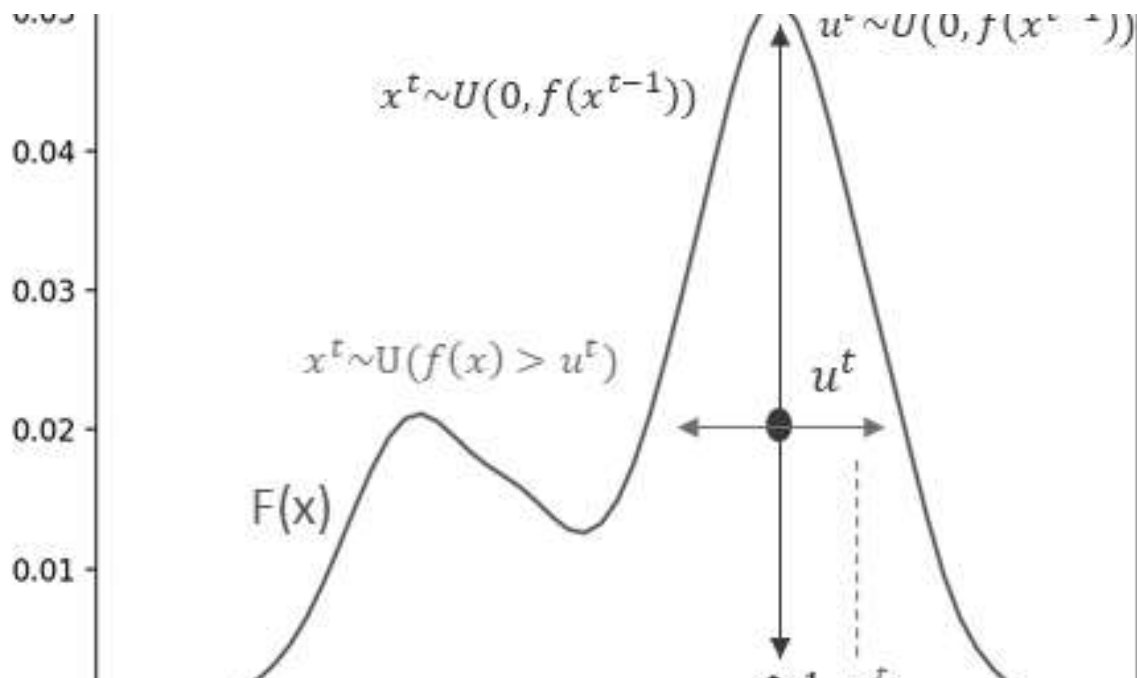


In R 語言自學系列 by Edward Tung

R語言自學日記(19)-介紹ARCH/GARCH模型

The introduction of ARCH-GARCH Model

Sep 7, 2018 🖱 184 💬 1



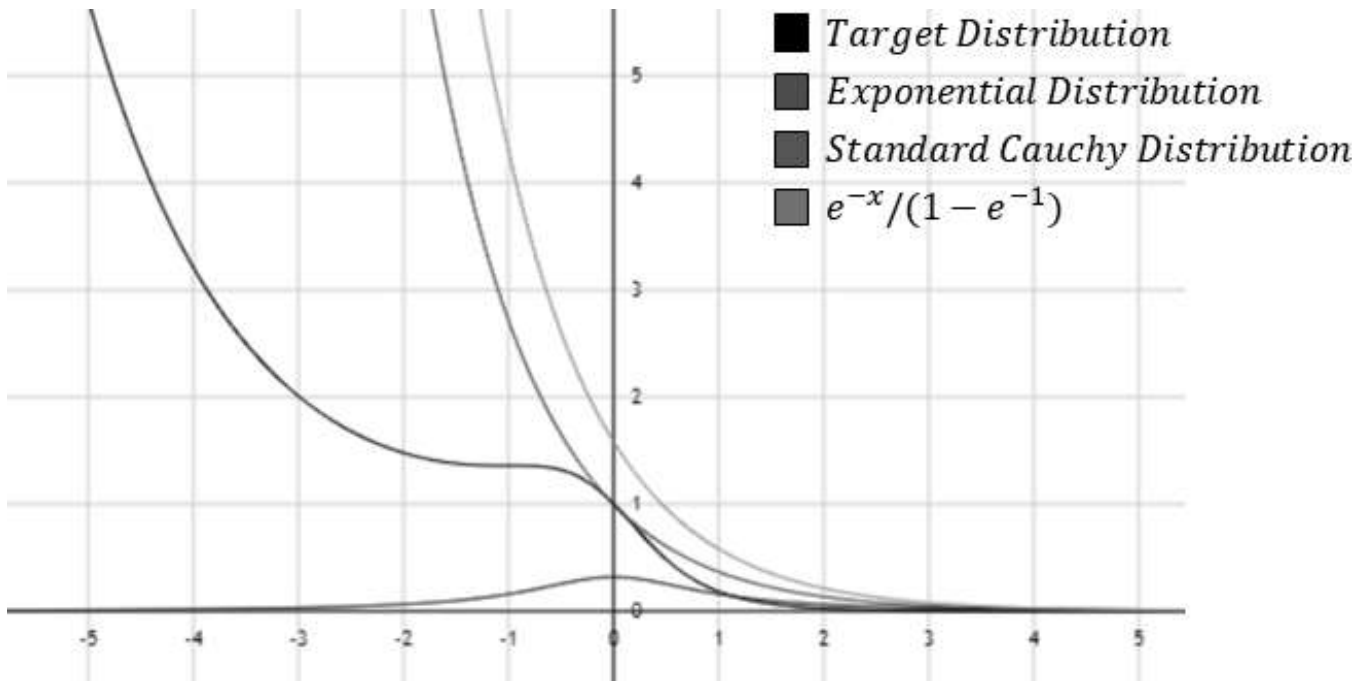
In 數學、人工智慧與蟒蛇 by Edward Tung

抽樣與蒙地卡羅(三)：馬可夫鏈蒙地卡羅方法

Sampling and Monte Carlo (03)：Markov Chain Monte Carlo (MCMC) Method

Feb 2, 2020 🖱 264 💬 1



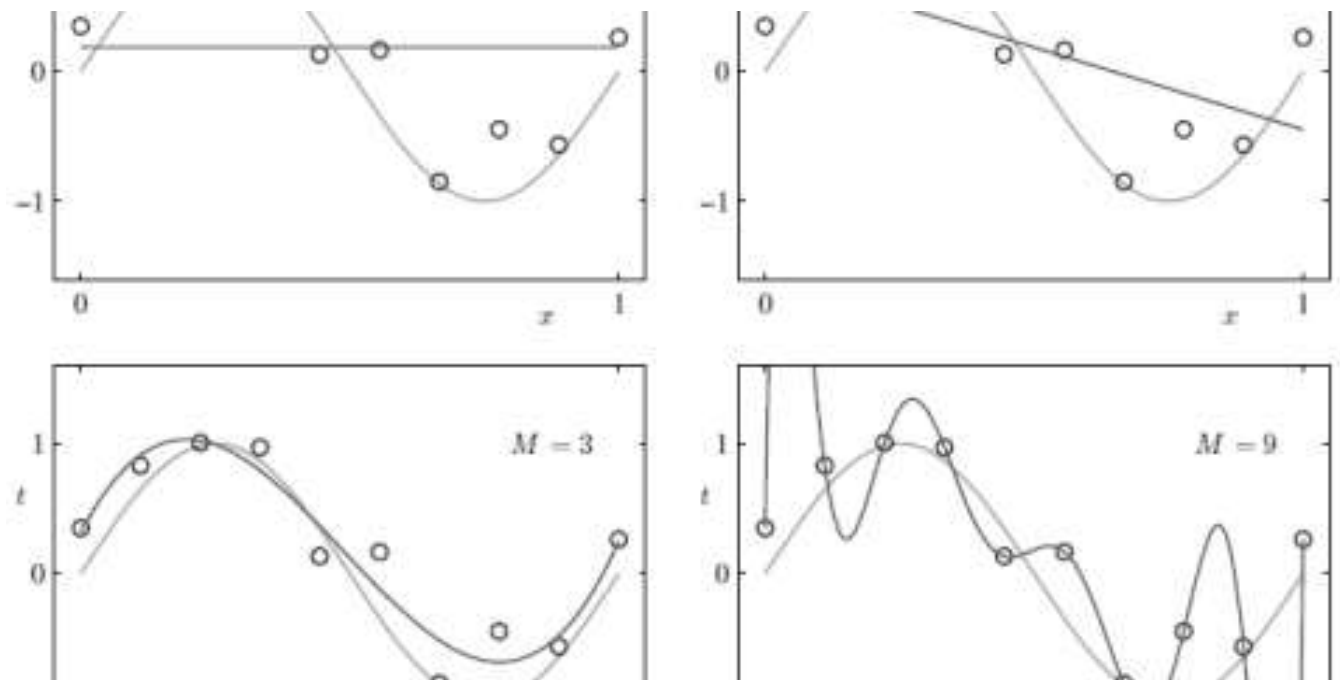


In 數學、人工智慧與蟒蛇 by Edward Tung

抽樣與蒙地卡羅(二)：蒙地卡羅方法與重要性抽樣

Sampling and Monte Carlo (02) : Monte Carlo Method & Importance Sampling

Jan 28, 2020 🖱 306



In R 語言自學系列 by Edward Tung

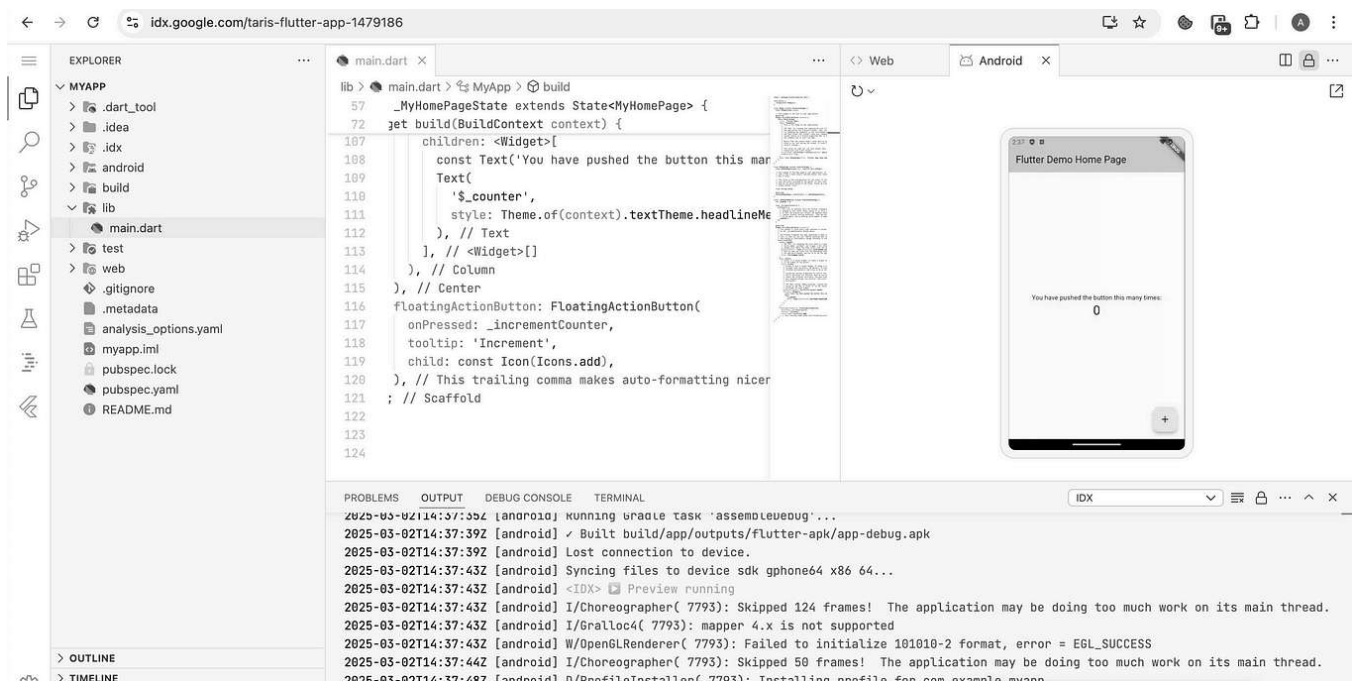
R語言自學日記(9)-迴歸模型介紹

線性迴歸、多變量迴歸與多項式迴歸

Jul 27, 2018 🖱 261 💬 2

[See all from Edward Tung](#)[See all from 數學、人工智慧與蟒蛇](#)

Recommended from Medium

 In Coding Beauty by Tari Ibaba

This new IDE from Google is an absolute game changer

This new IDE from Google is seriously revolutionary.

★ Mar 12 🖱 5.3K 💬 311



$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- Focuses on the correctness of positive predictions.

2. Recall:

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- Measures how well the model captures all positive instances.

3. F1 Score:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

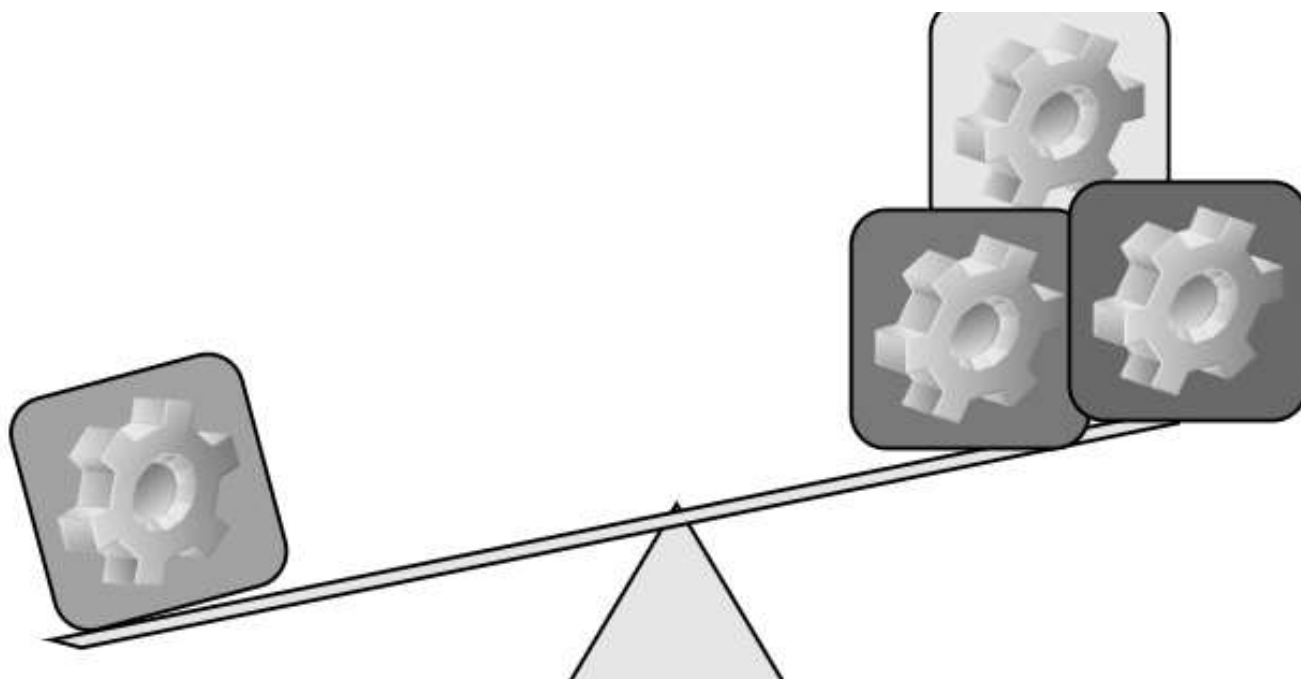


Piyush Kashyap

Understanding Precision, Recall, and F1 Score Metrics

In the world of machine learning, performance evaluation metrics play a critical role in determining the effectiveness of a model. Metrics...

Dec 2, 2024 🖱 13



Nermin Babalik

Handling Imbalanced Dataset in Deep Learning

Let's consider undersampling, oversampling, SMOTE and ensemble methods for combating imbalanced data sets with Deep Learning.

★ Jan 3 🖱 401 💬 2



 In Write A Catalyst by Adarsh Gupta

Google just Punished GeekforGeeks

And that's for a reason.

★ Apr 11 🖱 3.1K 💬 139



 In Hello, Love by Mariah

When You Stop Being Available, Everything Changes—Carl Jung

How withdrawing transforms your life.

★ Apr 24 🖱 10.1K 💬 268



💡 In ILLUMINATION by Dr. Nikita Singh

After Age 40, You Need to Stop Doing These Morning Habits—That Speed Up Aging

Your Morning Routine Making You Age Faster?

★ Apr 21 🖱 9.5K 💬 203



See more recommendations